



使用交叉验证和网格搜索优化 KNN 超参数

下面完整的实现代码，详细演示如何使用 `GridSearchCV` 和 `cross_val_score` 来寻找 KNN 模型的最佳超参数组合：

代码块

```
1  import numpy as np
2  import pandas as pd
3  import matplotlib.pyplot as plt
4  import seaborn as sns
5  from sklearn.datasets import load_iris
6  from sklearn.model_selection import train_test_split, GridSearchCV,
   RandomizedSearchCV, cross_val_score
7  from sklearn.preprocessing import StandardScaler
8  from sklearn.neighbors import KNeighborsClassifier
9  from sklearn.metrics import accuracy_score, classification_report,
   confusion_matrix
10 import warnings
11 warnings.filterwarnings('ignore')
12
13 # 设置中文显示
14 plt.rcParams['font.sans-serif'] = ['SimHei']
15 plt.rcParams['axes.unicode_minus'] = False
16
17 print("="*70)
18 print("🤖 KNN 模型超参数优化：交叉验证 + 网格搜索")
19 print("="*70)
20
21 # ===== 1. 加载和准备数据 =====
22 print("\n📁 步骤 1：加载数据")
23 print("-"*70)
24
25 # 加载鸢尾花数据集
26 iris = load_iris()
27 X = iris.data
28 y = iris.target
29
30 print(f"数据集大小：{X.shape[0]} 个样本，{X.shape[1]} 个特征")
31 print(f"类别：{iris.target_names}")
32
33 # 划分训练集和测试集
```

```

34 X_train, X_test, y_train, y_test = train_test_split(
35     X, y, test_size=0.2, random_state=42, stratify=y
36 )
37
38 # 数据标准化
39 scaler = StandardScaler()
40 X_train_scaled = scaler.fit_transform(X_train)
41 X_test_scaled = scaler.transform(X_test)
42
43 print(f"训练集: {X_train.shape[0]} 个样本")
44 print(f"测试集: {X_test.shape[0]} 个样本")
45
46 # ===== 2. 定义超参数网格 =====
47 print("\n⚙️ 步骤 2: 定义超参数网格")
48 print("-"*70)
49
50 # 定义要搜索的超参数
51 # KNeighborsClassifier 的关键参数:
52 # - n_neighbors: K 值 (邻居数量)
53 # - weights: 权重方式 (uniform 或 distance)
54 # - metric: 距离度量方式
55 # - p: 闵可夫斯基距离的参数 (p=1 曼哈顿, p=2 欧氏)
56
57 param_grid = {
58     'n_neighbors': list(range(1, 31)),          # K 值: 1-30
59     'weights': ['uniform', 'distance'],         # 权重方式
60     'metric': ['euclidean', 'manhattan', 'minkowski'], # 距离度量
61     'p': [1, 2]                                # 闵可夫斯基参数
62 }
63
64 # 计算总的参数组合数
65 total_combinations = (
66     len(param_grid['n_neighbors']) *
67     len(param_grid['weights']) *
68     len(param_grid['metric']) *
69     len(param_grid['p'])
70 )
71 print(f"总参数组合数: {total_combinations}")
72 print(f"\n超参数网格: ")
73 for key, value in param_grid.items():
74     print(f"    {key}: {value}")
75
76 # ===== 3. 使用 GridSearchCV 进行网格搜索 =====
77 print("\n🔍 步骤 3: 网格搜索 (GridSearchCV)")
78 print("-"*70)
79
80 # 创建基础 KNN 分类器

```

```

81 knn = KNeighborsClassifier()
82
83 # 创建 GridSearchCV 对象
84 # cv=5 表示 5 折交叉验证
85 # scoring='accuracy' 使用准确率作为评估指标
86 # n_jobs=-1 使用所有 CPU 核心加速
87 grid_search = GridSearchCV(
88     estimator=knn,          # 基础模型
89     param_grid=param_grid,  # 参数网格
90     cv=5,                  # 5 折交叉验证
91     scoring='accuracy',    # 评估指标
92     n_jobs=-1,             # 并行计算
93     verbose=1,             # 显示进度
94     return_train_score=True # 返回训练集分数
95 )
96
97 print("开始网格搜索...")
98 print("这将需要一些时间，请耐心等待...\n")
99
100 # 执行网格搜索
101 grid_search.fit(X_train_scaled, y_train)
102
103 print("\n✅ 网格搜索完成! ")
104
105 # ===== 4. 查看搜索结果 =====
106 print("\n 步骤 4: 查看搜索结果")
107 print("-"*70)
108
109 # 最佳参数组合
110 print("🏆 最佳超参数组合: ")
111 print(f"{'='*50}")
112 for param, value in grid_search.best_params_.items():
113     print(f"  {param}: {value}")
114 print(f"{'='*50}")
115
116 # 最佳得分
117 print(f"\n最佳交叉验证准确率: {grid_search.best_score_: .4f}
118 ({grid_search.best_score_*100: .2f}%)")
119
120 # 最佳模型
121 best_knn = grid_search.best_estimator_
122 print(f"\n最佳模型配置: ")
123 print(best_knn)
124
125 # ===== 5. 详细结果分析 =====
126 print("\n📊 步骤 5: 详细结果分析")
127 print("-"*70)

```

```

127
128 # 将搜索结果转换为 DataFrame
129 results_df = pd.DataFrame(grid_search.cv_results_)
130
131 # 按 mean_test_score 降序排列
132 results_df = results_df.sort_values('mean_test_score', ascending=False)
133
134 # 显示前 10 个最佳结果
135 print("📊 前 10 个最佳参数组合：")
136 print(f"{'='*100}")
137 print(f"{'排名':<5} {'准确率':<10} {'n_neighbors':<15} {'weights':<12} {'metric':<15} {'p':<5}")
138 print(f"{'-'*100}")
139
140 for i, row in results_df.head(10).iterrows():
141     print(f"{i+1:<7} {row['mean_test_score']:<12.4f} {row['param_n_neighbors']:<17} "
142           f"{row['param_weights']:<14} {row['param_metric']:<17} {row['param_p']:<7}")
143
144 print(f"{'='*100}")
145
146 # ===== 6. 可视化搜索结果 =====
147 print("\n 步骤 6: 可视化搜索结果")
148 print("-"*70)
149
150 fig, axes = plt.subplots(2, 2, figsize=(14, 10))
151
152 # 图 1: 不同 K 值的准确率
153 k_values = sorted(list(set(results_df['param_n_neighbors'])))
154 k_scores = []
155 for k in k_values:
156     subset = results_df[results_df['param_n_neighbors'] == k]
157     k_scores.append(subset['mean_test_score'].max())
158
159 axes[0, 0].plot(k_values, k_scores, marker='o', linewidth=2, markersize=8)
160 axes[0, 0].set_xlabel('K Value (n_neighbors)', fontsize=12)
161 axes[0, 0].set_ylabel('Accuracy', fontsize=12)
162 axes[0, 0].set_title('K Value vs Best Accuracy', fontsize=14)
163 axes[0, 0].grid(True, alpha=0.3)
164 axes[0, 0].axvline(x=grid_search.best_params_['n_neighbors'],
165                    color='red', linestyle='--', label=f'Best K={grid_search.best_params_["n_neighbors"]}')
166 axes[0, 0].legend()
167
168 # 图 2: 权重方式对比

```

```

169 weights_comparison = results_df.groupby('param_weights')
    ['mean_test_score'].mean()
170 axes[0, 1].bar(weights_comparison.index, weights_comparison.values,
171                color=['blue', 'orange'], alpha=0.7)
172 axes[0, 1].set_xlabel('Weights', fontsize=12)
173 axes[0, 1].set_ylabel('Average Accuracy', fontsize=12)
174 axes[0, 1].set_title('Weights Comparison', fontsize=14)
175 axes[0, 1].grid(True, alpha=0.3, axis='y')
176
177 # 图 3: 不同距离度量的准确率
178 metric_comparison = results_df.groupby('param_metric')
    ['mean_test_score'].mean()
179 axes[1, 0].bar(metric_comparison.index, metric_comparison.values,
180                color=['green', 'purple', 'red'], alpha=0.7)
181 axes[1, 0].set_xlabel('Metric', fontsize=12)
182 axes[1, 0].set_ylabel('Average Accuracy', fontsize=12)
183 axes[1, 0].set_title('Distance Metric Comparison', fontsize=14)
184 axes[1, 0].grid(True, alpha=0.3, axis='y')
185
186 # 图 4: 训练集 vs 验证集准确率 (检测过拟合)
187 train_scores = results_df['mean_train_score']
188 test_scores = results_df['mean_test_score']
189 axes[1, 1].scatter(range(len(train_scores)), train_scores,
190                    alpha=0.5, label='Train Score', s=30)
191 axes[1, 1].scatter(range(len(test_scores)), test_scores,
192                    alpha=0.5, label='Test Score', s=30)
193 axes[1, 1].set_xlabel('Parameter Combination Index', fontsize=12)
194 axes[1, 1].set_ylabel('Accuracy', fontsize=12)
195 axes[1, 1].set_title('Train vs Test Score', fontsize=14)
196 axes[1, 1].legend()
197 axes[1, 1].grid(True, alpha=0.3)
198
199 plt.tight_layout()
200 plt.show()
201
202 # ===== 7. 使用最佳模型进行预测和评估 =====
203 print("\n 步骤 7: 使用最佳模型评估")
204 print("-"*70)
205
206 # 在测试集上评估最佳模型
207 y_pred = best_knn.predict(X_test_scaled)
208
209 accuracy = accuracy_score(y_test, y_pred)
210 print(f"

```

```

214
215 # 详细分类报告
216 print(f"\n 分类报告: ")
217 print(classification_report(y_test, y_pred, target_names=iris.target_names))
218
219 # 混淆矩阵
220 cm = confusion_matrix(y_test, y_pred)
221 print(f"\n 混淆矩阵: ")
222 print(cm)
223
224 # ===== 8. 交叉验证稳定性分析 =====
225 print("\n 步骤 8: 交叉验证稳定性分析")
226 print("-"*70)
227
228 # 使用最佳模型进行多次交叉验证
229 cv_scores = cross_val_score(best_knn, X_train_scaled, y_train, cv=10,
                               scoring='accuracy')
230
231 print(f"10 折交叉验证结果: ")
232 print(f"{'='*50}")
233 print(f"各折准确率: {cv_scores}")
234 print(f"平均准确率: {cv_scores.mean():.4f} ({cv_scores.mean()*100:.2f}%)")
235 print(f"标准差: {cv_scores.std():.4f}")
236 print(f"{'='*50}")
237
238 # ===== 9. 随机搜索对比 (可选) =====
239 print("\n🤖 步骤 9: 随机搜索对比 (RandomizedSearchCV)")
240 print("-"*70)
241
242 # 定义随机搜索的参数分布
243 param_dist = {
244     'n_neighbors': list(range(1, 31)),
245     'weights': ['uniform', 'distance'],
246     'metric': ['euclidean', 'manhattan', 'minkowski'],
247     'p': [1, 2]
248 }
249
250 # 创建随机搜索对象
251 random_search = RandomizedSearchCV(
252     estimator=KNeighborsClassifier(),
253     param_distributions=param_dist,
254     n_iter=50, # 随机尝试 50 次
255     cv=5,
256     scoring='accuracy',
257     n_jobs=-1,
258     random_state=42,
259     verbose=1

```

```

260 )
261
262 print("开始随机搜索...")
263 random_search.fit(X_train_scaled, y_train)
264
265 print(f"\n随机搜索结果: ")
266 print(f"最佳参数: {random_search.best_params_}")
267 print(f"最佳准确率: {random_search.best_score_:.4f}
    ({random_search.best_score_*100:.2f}%)")
268
269 # 对比网格搜索和随机搜索
270 print(f"\n 方法对比: ")
271 print(f"{'='*50}")
272 print(f"网格搜索最佳准确率: {grid_search.best_score_:.4f}")
273 print(f"随机搜索最佳准确率: {random_search.best_score_:.4f}")
274 print(f"{'='*50}")
275
276 # ===== 10. 预测新样本 =====
277 print("\n🧠 步骤 10: 预测新样本")
278 print("-"*70)
279
280 # 新的未知样本
281 new_samples = np.array([
282     [5.1, 3.5, 1.4, 0.2],
283     [6.2, 3.4, 5.4, 2.3],
284     [5.9, 3.0, 4.2, 1.5]
285 ])
286
287 # 标准化新样本
288 new_samples_scaled = scaler.transform(new_samples)
289
290 # 预测
291 predictions = best_knn.predict(new_samples_scaled)
292 probabilities = best_knn.predict_proba(new_samples_scaled)
293
294 print("🌸 新样本预测结果: ")
295 print(f"{'='*70}")
296 print(f"{'样本':<6} {'特征':<30} {'预测类别':<15} {'置信度':<10}")
297 print(f"{'-'*70}")
298
299 for i, (sample, pred, probs) in enumerate(zip(new_samples, predictions,
    probabilities)):
300     pred_name = iris.target_names[pred]
301     confidence = probs[pred]
302     features_str = f"[{sample[0]:.1f}, {sample[1]:.1f}, {sample[2]:.1f},
    {sample[3]:.1f}]"
303     print(f"{i+1:<8} {features_str:<32} {pred_name:<18} {confidence*100:.2f}%")

```

```
304
305 print(f"{'='*70}")
306
307 print("\n" + "="*70)
308 print("✅ 超参数优化任务完成! ")
309 print("="*70)
310
311 # 保存最佳模型
312 import joblib
313 joblib.dump(best_knn, 'best_knn_model.pkl')
314 joblib.dump(scaler, 'scaler.pkl')
315 print("\n📁 最佳模型和标准化器已保存! ")
```

关键概念解析

1. 网格搜索 (GridSearchCV)

工作原理：

- 穷举所有可能的参数组合
- 对每种组合进行交叉验证
- 选择表现最好的参数组合

优点：

- 保证找到最优解（在给定网格内）
- 结果可重复

缺点：

- 计算量大，耗时长
- 参数组合呈指数级增长

2. 随机搜索 (RandomizedSearchCV)

工作原理：

- 随机采样参数组合
- 只尝试指定数量的组合
- 更快但不保证最优

使用场景：

- 参数空间很大时
- 需要快速获得近似最优解

 3. 交叉验证 (Cross-Validation)

5 折交叉验证流程：

代码块

```
1 数据分为 5 份：
2 折 1：[测试][训练][训练][训练][训练]
3 折 2：[训练][测试][训练][训练][训练]
4 折 3：[训练][训练][测试][训练][训练]
5 折 4：[训练][训练][训练][测试][训练]
6 折 5：[训练][训练][训练][训练][测试]
7
8 最终得分 = 5 次测试得分的平均值
```

核心超参数说明

参数	说明	推荐范围
n_neighbors	K 值，邻居数量	1-30（奇数更好）
weights	权重方式	'uniform'（均匀） 或'distance'（距离加权）

metric	距离度量	'euclidean'（欧氏）、'manhattan'（曼哈顿）
p	闵可夫斯基参数	p=1（曼哈顿）、p=2（欧氏）

实用技巧

1. 加速网格搜索

代码块

```
1  # 使用并行计算
2  grid_search = GridSearchCV(..., n_jobs=-1)
3
4  # 减少参数组合
5  param_grid = {
6      'n_neighbors': [3, 5, 7, 9, 11], # 而不是 range(1, 31)
7      'weights': ['uniform', 'distance']
8  }
```

2. 自定义评估指标

代码块

```
1  from sklearn.metrics import make_scorer, f1_score
2
3  # 使用 F1 分数作为评估指标
4  grid_search = GridSearchCV(
5      ...,
6      scoring=make_scorer(f1_score, average='weighted')
7  )
```

3. 保存和加载最佳模型

代码块

```
1  import joblib
2
3  # 保存
4  joblib.dump(best_knn, 'best_knn_model.pkl')
5
6  # 加载
7  loaded_model = joblib.load('best_knn_model.pkl')
```

运行输出示例

代码块

```
1  🏆 最佳超参数组合:
2  =====
3      n_neighbors: 7
4      weights: distance
5      metric: euclidean
6      p: 2
7  =====
8
9  最佳交叉验证准确率: 0.9750 (97.50%)
10
11  前 10 个最佳参数组合:
12  =====
13  =====
14  排名      准确率      n_neighbors      weights      metric      p
15  -----
16  1          0.9750          7          distance      euclidean      2
17  2          0.9750          9          distance      euclidean      2
18  3          0.9667          5          distance      euclidean      2
19  ...
```

这段代码完整演示了如何使用交叉验证和网格搜索来优化 KNN 模型的超参数! ✨